Hydroficient IoT Cyber Defense
Project 2: Python for IoT Security

Basics of Python

Setting Up the Python Environment
Setting Up Google Colab
Step 1: Open Google Colab
Go to colab.research.google.com
Step 2: Create a New Notebook
When you open Colab, a popup will appear. Do not close it. At the bottom of the popup, you will see **"New Notebook."** Click on it.
You'll see a blank page with a box. That box is called a **cell** — it's where you write code.
Step 3: Write Your First Line of Code
Click inside the cell and type:

```
print("Connected to GM-HYDROLOGIC-01")
```

Step 4: Run the Code
Press **Shift + Enter** (or click the play button on the left side of the cell).
You should see: Connected to GM-HYDROLOGIC-01
Recap:
Python = Easy & Powerful → A beginner-friendly language for AI, data, and automation.
How to Use? → For this externship, we will use Google Colab to write and run Python code online.
Why Python Here? → Essential for IoT security analysis and building monitoring tools.
Key Strengths of Python → Simple, rich libraries, strong community support.

Know the Python Essentials
Quick Recap: Basic Python Grammar

Concept	What It Does	How You'll Use It
Variables	Store data for later use	Store sensor readings, device IDs, thresholds
Data Types	Define what kind of data (int, float, str, bool)	Pressure is a float, device_id is a string, is_leak is a boolean
Operators	Calculate and compare values	Calculate pressure drop, check if flow exceeds threshold

Conditionals	Make decisions based on conditions	Trigger alerts when readings are abnormal
Loops	Repeat actions across multiple items	Process hundreds of sensor readings
Functions	Create reusable blocks of code	Build `check_leak()` or `analyze_reading()` functions
Lists	Store ordered collections	Hold multiple readings over time
Dictionaries	Store key-value pairs	Match the JSON format HYDROLOGIC devices use

Know About Data Structures in Python
Quick Recap

Data Structure	Use When...	Hydro Example
List	You need ordered, changeable collection	Storing multiple flow readings
Tuple	Data should never change	Device configuration, coordinates
Set	You need unique items, order doesn't matter	Tracking which devices are active
Dictionary	You need key-value pairs	Single sensor reading with multiple fields

Task: Practice Creating Data Structures
Follow the Steps Below
Open the Notebook
Click the link below to open the Data Structures practice notebook
Make a copy: **File → Save a copy in Drive**
Rename your copy: [Your Name] - Data Structures Practice
Work Through the Examples
The notebook covers Lists, Tuples, Sets, Dictionaries, and Slicing/Indexing
Run each code cell to see how the examples work
Experiment in the blank cells — try adding, removing, and modifying elements
Complete the Exercises
At the end of the notebook, you'll find 5 exercises:
Exercise 1: List Operations (fruits)
Exercise 2: Tuple Operations (numbers)
Exercise 3: Set Operations (union, intersection, difference)
Exercise 4: Dictionary Operations (book info)

Exercise 5: Slicing and Indexing (numbers 1-20)
Complete all 5 exercises in the provided code cells
Make sure each cell runs without errors

Solution: https://colab.research.google.com/drive/1wBI4mdoV7oOGowWyWLsCTe3pI_mo0gGb?usp=sharing

What is Control Flow in Python
Quick Recap:

Structure	Type	What It Does
Sequential	—	Execute code line by line
If	Conditional	Execute if condition is true
If-Else	Conditional	Choose between two paths
If-Elif-Else	Conditional	Choose between multiple paths
For Loop	Loop	Iterate over a known sequence
While Loop	Loop	Repeat while condition is true
Break	Jump	Exit loop immediately
Continue	Jump	Skip to next iteration
Pass	Jump	Do nothing (placeholder)

Task: Practice Control Flow Exercises
Follow the Steps Below
Open the Notebook
Click the link below to open the Control Flow practice notebook
Make a copy: **File → Save a copy in Drive**
Rename your copy: [Your Name] - Control Flow Practice
Work Through the Examples
The notebook covers Decision-Making Statements, Loops, and Jump Statements
Run each code cell to see how the examples work
Experiment in the blank cells — try changing values and conditions to see what happens
Pay attention to how different statements alter the program's execution:
What happens if you use continue in a loop?

What happens if you use break instead?
Try nesting an if inside a for loop — what can you build?
Complete the Exercises
At the end of the notebook, you'll find 6 exercises:
Exercise 1: Even or Odd (conditionals)
Exercise 2: Guess the Number (while loop + conditionals)
Exercise 3: Print Prime Numbers (for loop + conditionals)
Exercise 4: Factorial Calculation (while loop)
Exercise 5: Multiplication Table (for loop)
Exercise 6: Count Vowels in a String (for loop + continue)
Complete all 6 exercises in the provided code cells
Make sure each cell runs without errors
My Solution: https://colab.research.google.com/drive/1zeZq6lK1nCcJw9On9iM89tPqhMMrQosR?usp=sharing

Know What Are Functions in Python
Quick Recap:
Functions are reusable code blocks that perform specific tasks—like Amazon teams handling payments or shipping.
There are four types based on input/output: nothing in/out, input only, output only, and both input and output.
Functions work with **data structures** (storage) and **control flow** (logic) to create efficient, organized programs.

Task: Practice Performing Functions
Follow the Steps Below
Open the Notebook
Click the link below to open the Functions practice notebook
Make a copy: **File → Save a copy in Drive**
Rename your copy: [Your Name] - Functions Practice
Work Through the Examples
The notebook covers all four types of functions:
Nothing goes in, nothing comes out
Nothing goes in, something comes out
Something goes in, nothing comes out
Something goes in, something comes out
Run each code cell to see how the examples work
Complete the Exercises
At the end of the notebook, you'll find 6 exercises:
Exercise 1: Convert Temperature (Celsius to Fahrenheit)
Exercise 2: Check Even or Odd
Exercise 3: Find Maximum of Two Numbers
Exercise 4: Calculate Factorial
Exercise 5: Reverse a String

Exercise 6: Count Vowels in a String
My Solution: https://colab.research.google.com/drive/1Sn8T3sUak9KZbAlRFwYzX7VS_z64Z35T?usp=sharing

Task: On Google Colab, Provide An Input To Generate An Output (Mandatory)
Follow the Steps Below
Open the Notebook
Click the link below to open the Python Assignment notebook
Make a copy immediately: File → Save a copy in Drive
Rename your copy: [Your Name] - Python Assignment
Complete the 4 Questions
Test Your Code
Each question has example tests — run them to check your work
Make sure your output matches the expected output

My Solution: https://colab.research.google.com/drive/17ldWCdbNEMazrxHFJTXxZFtZtDq07BZl?usp=

Prepare Your Data for Analysis
In the real world, data is messy. Sensor readings have missing values. Timestamps come in different formats. Device IDs are inconsistent. Before you can analyze data or detect threats, you need to **clean and prepare** it.

Know About Data Preparation Concepts
What is Data Preparation?
Data preparation is all about gathering, cleaning, organizing, and transforming raw data into a format that's easy to use for analysis.

This process includes:

Concept	What It Does	Why It's Needed	Example
Cleaning	Identifies and corrects **errors, missing values, duplicates, and outliers**.	Ensures data **accuracy, consistency, and completeness** before analysis.	Removing duplicate sensor readings, filling in missing timestamps, fixing inconsistent pressure units (PSI vs bar).

Transforming	Converts data into a **usable format** through **encoding, scaling, and feature engineering**.	Standardizes data for analysis and **makes it machine-readable**.	Converting timestamps to a standard format, normalizing pressure values to the same unit, encoding device status as numbers.
Integration	Combines **multiple datasets** into a unified format.	Allows analysis across different data sources and ensures consistency.	Combining sensor logs from multiple HYDROLOGIC devices, merging device metadata with reading history.
Splitting	Divides data into **training, validation, and testing sets**.	Prevents bias by ensuring the model **learns and generalizes well**.	Splitting sensor data into training and test sets for an anomaly detection model.

The goal of this preparation is to ensure you get the data in an optimized form so that you can get the best result while doing the analysis.

Why is Data Preparation Critical for IoT Security?
IoT sensor data can be messy — missing timestamps, inconsistent units, duplicate readings, or values outside normal ranges. Without proper preparation, security tools might miss real threats or generate false alarms.

Structured vs. Unstructured Data in Data Preparation
Data preparation isn't the same for every type of dataset.

Structured Data (like spreadsheets, CSVs, databases) follows a fixed format (rows & columns).	**Semi-structured Data** (like JSON files, tabular PDFs) requires **extra steps** like **handling formatting inconsistencies and dealing with missing data** before analysis.

Scenario 1: Structured Data
Let's say I send you a CSV file containing a week of HYDROLOGIC sensor readings — each row represents a reading, with columns for device_id, timestamp, pressure_upstream, pressure_downstream, and flow_rate. Since the data is

structured, preparing it is straightforward:
Fixing missing values – Some pressure readings are missing, so you identify gaps and decide whether to flag them or interpolate from nearby readings.
Standardizing numerical values – Some pressure values are in different units (PSI vs bar), so you convert them all to PSI.
Cleaning text inconsistencies - Device IDs appear in mixed formats ("GM-HYDROLOGIC-01" vs "gm_hydrologic_01"), so you standardize them.
Once cleaned, this dataset is ready to analyze for anomalies, trends, and potential security issues.

Scenario 2: Semi-Structured Data (JSON Logs)
Now, let's say I send you raw JSON logs directly from the HYDROLOGIC devices. Unlike the clean CSV, this data is nested and requires parsing:
Extracting nested data – Each device record contains an array of readings that need to be flattened for analysis.
Handling inconsistent formatting – Some timestamps use "2024-01-15T14:30:00Z" while others use "01/15/2024 2:30 PM", requiring format standardization.
Validating data types – Some numeric values arrived as strings ("85.2" instead of 85.2), requiring conversion.
Identifying missing fields – Some readings are missing pressure values, requiring you to flag or handle them appropriately.

Quick Reflection
You're working on a HYDROLOGIC sensor dataset, and the data is messy:
🔷 Some readings are **missing timestamps** or pressure values.
🔷 There are **duplicate readings** where the same sensor message appears twice.
🔷 Timestamp formats are **inconsistent** (some in "MM/DD/YYYY", others in "YYYY-MM-DD").
Question: What are the **first three steps you would take** to clean and organize this data?
Answer (click here to expand)
Handle missing values – Flag readings with missing timestamps or pressure values. Decide whether to drop them or interpolate.
Remove duplicates – Identify and drop repeated sensor messages (common with network retries).
Standardize timestamp formats – Convert all timestamps to ISO format (YYYY-MM-DDTHH:MM:SSZ).
Build a Mock Sensor

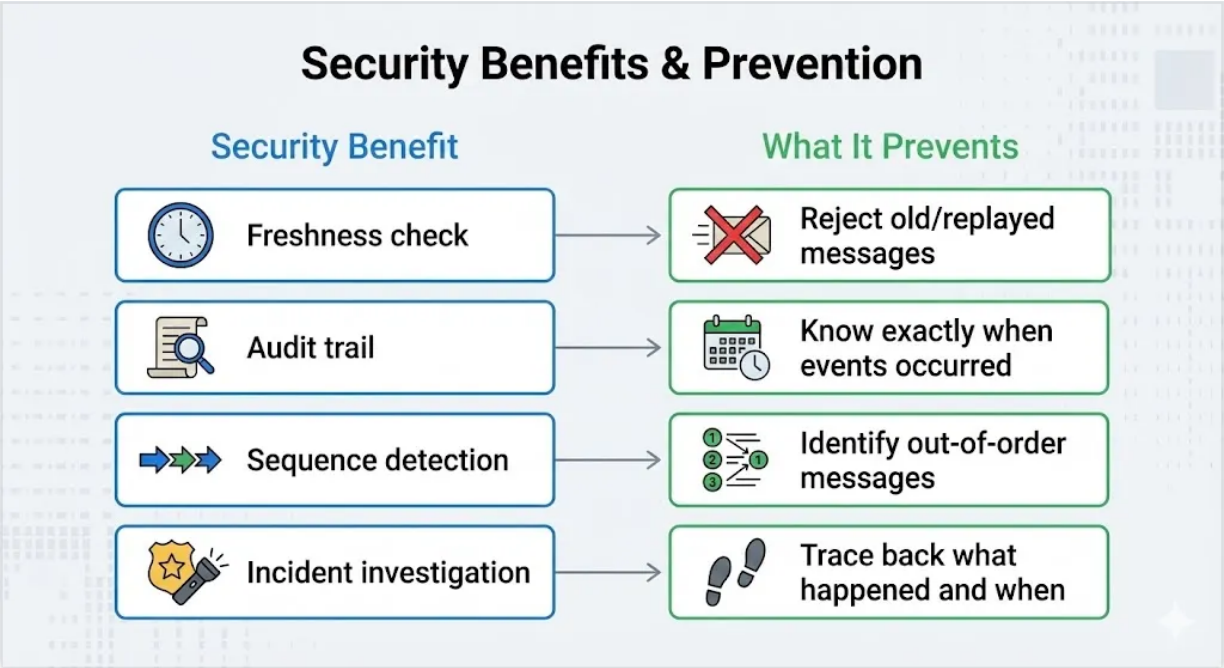
Why Timestamps and Counters Matter for IoT Security

The Envelope Analogy: Why Every Message Needs a Date
Imagine you receive a letter in the mail. One of the first things you check is the

date it was sent. Why? Because knowing *when* something was sent tells you a lot:
Is this letter still relevant, or is it outdated?
Did I receive this in the right order?
Is someone trying to trick me with an old letter?
Now imagine a letter with **no date at all**. How would you know if it was sent yesterday or last year? You couldn't. And that's exactly the problem with IoT messages that don't include timestamps.
IoT sensors work the same way. Every message a sensor sends needs:
A **timestamp** — *when* it was sent
A **counter** — *which number* in the sequence it is
Without these two pieces of metadata, bad actors could wreak havoc on your system.

Imagine a hacker captures a legitimate "shut off water" command from last week. Without timestamps or counters, they could **resend that old command** whenever they want, causing water shutoffs at random times. This is called a **replay attack**, and it's one of the most common threats to IoT systems.

What Are Timestamps?
A **timestamp** is simply a record of when something happened. In IoT, every sensor reading includes a timestamp so we know exactly when that measurement was taken.



Working with Timestamps in Python
Python has a built-in module called datetime that makes working with timestamps easy. IoT systems typically use **ISO 8601 format** for timestamps because it's a universal standard that works across different systems and time zones.

What Are Counters?

A **counter** (or sequence number) is simply a number that increases by 1 with each message. Think of it like numbering the pages in a book—if page 5 is missing, you know something's wrong!

Why Counters Matter for Security

Counter Value	What It Means
Expected: 42, Got: 42	✅ Message is valid
Expected: 42, Got: 41	⚠️ Duplicate/replayed message
Expected: 42, Got: 44	⚠️ Messages 42-43 are missing
Expected: 42, Got: 100	🚨 Something very wrong

💡 **Timestamps + Counters = Double Protection**

Timestamps tell you *when* a message was created

Counters tell you *which* message it is in the sequence

Together, they make replay attacks much harder to execute!

Putting It All Together: Hydroficient Sensor Message

Here's what a complete sensor message looks like with timestamps and counters:

```
{
  "device_id": "GM-HYDROLOGIC-01",
  "timestamp": "2024-01-15T14:30:45.123456+00:00",
  "counter": 42,
  "pressure_upstream": 85.2,
  "pressure_downstream": 78.4,
  "flow_rate": 37.5
}
```

Each field serves a purpose:

Field	Purpose
device_id	Identifies which sensor sent this
timestamp	When the reading was taken (UTC)
counter	Sequence number for replay detection
pressure_upstream	Actual sensor data
pressure_downstream	Actual sensor data
flow_rate	Actual sensor data

Quick Recap

Why Timestamps?

Record when events happen

Enable freshness checks (reject old messages)

Create audit trails for investigations

Why Counters?

Track message sequence
Detect missing or duplicate messages
Prevent replay attacks
Python Tools
datetime.now(timezone.utc) → Get current UTC time
.isoformat() → Convert to ISO 8601 string
Simple counter += 1 → Track sequence
Security Impact
Timestamps + Counters make replay attacks detectable
Without them, attackers could resend old commands
Critical for protecting water shutoff systems!

Simulating Realistic Sensor Data
Why Create Fake Data? Because Real Sensors Cost Thousands!
Real water sensors like Hydroficient's HYDROLOGIC devices cost thousands of dollars and are installed in buildings across the country. We can't just grab one to experiment with. Instead, we create **mock data**—realistic fake sensor readings that let us:
Test our code without expensive hardware
Experiment safely without affecting real systems
Create edge cases (like leaks or attacks) that rarely happen in real life
Learn patterns in data before working with production systems

Understanding Water Sensor Values
Before we can simulate data, we need to understand what realistic values look like. Hydroficient's HYDROLOGIC sensors measure three key things:

Measurement	What It Means	Normal Range	Unit
Pressure Upstream	Water pressure before the valve	75-90	PSI
Pressure Downstream	Water pressure after the valve	40 - 60	PSI
Flow Rate	How fast water is flowing	30-50	gallons/min

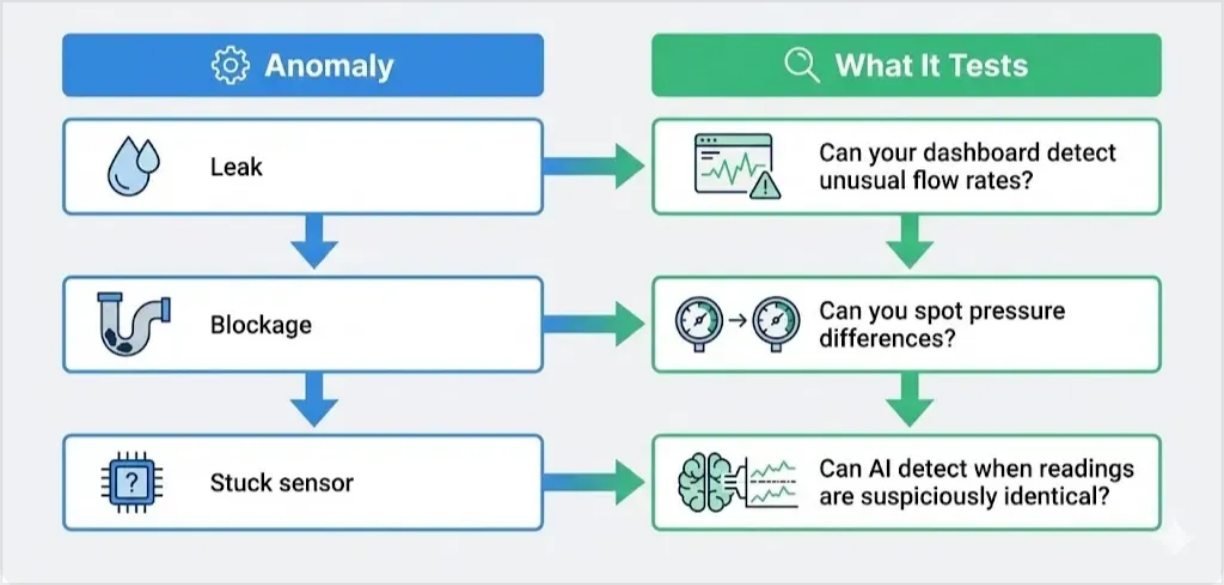
💡 **Why Two Pressure Readings?** The difference between upstream and downstream pressure can indicate problems:
Upstream high, downstream low → Possible blockage
Both pressures nearly equal → Valve might be fully open
Both pressures dropping → Supply issue or major leak

Python's Random Module
Python's built-in random module lets us generate random numbers—perfect for simulating sensor readings that vary realistically.
Key Functions:

random.uniform(a, b) → Random float between a and b
random.randint(a, b) → Random integer between a and b
random.choice(list) → Pick one item from a list
random.gauss(mean, std) → Random number with normal distribution
Sometimes you want the "random" numbers to be the same every time (useful for testing). Use random.seed().
Why 42? It's a common convention (a reference to *The Hitchhiker's Guide to the Galaxy*). Any number works — what matters is that the same seed produces the same sequence of "random" numbers.

Simulating Realistic Sensor Values
Real sensors don't jump wildly between values—they change gradually. A pressure reading of 82.3 PSI is likely to be followed by something close, like 82.1 or 82.5. Better approach—start with a base value and add small variations: Add small variation (±2-3 units) for realism

Creating Anomalies and Building the Complete Sensor
Why will you create these anomalies?
Because **you can't defend against something you can't simulate.**



The Complete WaterSensor Class: https://colab.research.google.com/drive/1_y1O-YyrorPbGD48rbmZAO-XdtTWe9Nt?usp=sharing

Component	What It Does	Why It Matters
__init__	Sets up device ID, counter, base values	Each sensor has its own identity and state

counter	Increments with every reading	Enables replay attack detection
timestamp	Records when reading was generated	Proves message freshness
base values + variation	Creates realistic drift	Makes data look authentic
get_leak_reading()	Simulates high flow	Tests leak detection
get_blockage_reading()	Simulates pressure differential	Tests blockage detection
get_stuck_reading()	Simulates sensor malfunction	Tests stuck sensor detection

Task: Build Your Sensor Log Generator (Mandatory)

Build Your WaterSensor Class
Your sensor generator must:
Create a WaterSensor Class
Your WaterSensor class should include:
device_id - A unique identifier for the sensor
counter - A sequence number that increments with each reading
A method to generate readings with:
device_id
timestamp (ISO 8601 format, UTC)
counter
pressure_upstream (normal range: 75-90 PSI)
pressure_downstream (normal range: 70-85 PSI)
flow_rate (normal range: 30-50 gallons/min)
Generate 100 Readings
Your output file should contain:
97 normal readings with realistic variation
3 anomalous readings (at least one of each type):
Leak (high flow rate)
Blockage (pressure difference)
Stuck sensor (identical values)
Verify Your Output
First reading has counter: 1
Last reading has counter: 100
All readings have valid ISO 8601 timestamps
Pressure values are in realistic ranges (70-95 PSI)
Normal flow rates are between 30-50 LPM
Leak readings have flow rates between 80-120 LPM

Blockage readings have high upstream/low downstream pressure
Stuck readings have identical values

Save to JSON File:
Save all readings to a file called sensor_data.json with proper formatting (use indent=2).
Tip 1: Use round(value, 1) to keep readings to one decimal place—it looks more realistic!
Tip 2: Don't forget to import timezone from datetime for UTC timestamps: from datetime import datetime, timezone
Tip 3: Use random.shuffle() to mix your anomalies into the normal readings so they're not all at the end.
Tip 4: Test your counter by printing the first and last readings—the counters should be 1 and 100!